What is Object-Oriented Programming, and How Does Encapsulation Work?

Object-oriented programming, also known as OOP, is a programming style in which developers treat everything in their code like a real-world object.

A class is like a blueprint for creating objects. Every single object created from a class has attributes that define data and methods that define the behaviors of the objects.

In a previous lesson, you learned how to create classes. Here's a reminder of the syntax:

```
class ClassName:
    def __init__(self, parameters):
        attribute = value


    def method_name(self):
        # method logic
```

Here's an example of a class that uses the `__init__` special method to initialize the brand and color attributes whenever an object is created using the class:

```
class Car:
    def __init__(self, brand, color):
        self.brand = brand
        self.color = color


# create two objects from the Car class
car1 = Car('Toyota', 'red')
car2 = Car('Lambo', 'green')


print('Car 1 Brand:', car1.brand) # Car 1 Brand: Toyota
```

```
print('Car 1 Color:', car1.color) # Car 1 Color: red


print('Car 2 Brand:', car2.brand) # Car 2 Brand: Lambo
print('Car 2 Color:', car2.color) # Car 2 Color: green
```

Object-oriented programming has four key principles that help you organize and manage code effectively. They are encapsulation, inheritance, polymorphism, and abstraction.

The rest of this lesson will focus on how encapsulation works.

Encapsulation is the bundling of the attributes and methods of an object into a single unit, the class.

With encapsulation, you can hide the internal state of the object behind a simple set of public methods and attributes that act like doors. Behind those doors are private attributes and methods that control how the data changes and who can see it.

Let's say you want to track a wallet balance. You want to allow people to deposit or withdraw money from the wallet, but no one should be able to tamper with the balance directly.

In that case, you can make `deposit()` and `withdraw()` public methods, and you hide the balance under the `_balance` attribute:

```
class Wallet:
    def __init__(self, balance):
        self._balance = balance # For internal use by
convention


    def deposit(self, amount):
        if amount > 0:
            self._balance += amount # Add to the balance safely


    def withdraw(self, amount):
```

```
        if 0 < amount <= self._balance:
            self._balance -= amount # Remove from the balance safely
```

By convention, prefixing attribute and methods with a single underscore means they are meant for internal use. No one should directly access them from outside the class since it defies the principles of encapsulation, which can lead to bugs.

While a single underscore prefix is just a convention, prefixing attributes and methods with a double underscore effectively prevents them to be accessed from the outside of their class, making those attributes and methods private.

```
class Wallet:
    def __init__(self, balance):
        self.__balance = balance # Private attribute

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount # Add to the balance safely

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount # Remove from the balance safely

account = Wallet(500)
print(account.__balance) # AttributeError: 'Wallet' object has no attribute '__balance'
```

To get the current value of `__balance`, you can define a get_balance method. For example:

```
class Wallet:
```

```
    def __init__(self, balance):
        self.__balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount

    def get_balance(self):
        return self.__balance


acct_one = Wallet(100)
acct_one.deposit(50)
print(acct_one.get_balance()) # 150

acct_two = Wallet(450)
acct_two.withdraw(28)
print(acct_two.get_balance()) # 422

acct_two.deposit(150)
print(acct_two.get_balance()) # 572
```

You can also define a private `__validate` method to check if every deposit or withdrawal amount is a positive number:

```
class Wallet:
    def __init__(self):
```

```
        self.__balance = 0

    def __validate(self, amount):
        if amount < 0:
            raise ValueError('Amount must be positive')

    def deposit(self, amount):
        self.__validate(amount)
        self.__balance += amount

    def withdraw(self, amount):
        self.__validate(amount)
        if amount > self.__balance:
            raise ValueError('Insufficient funds')
        self.__balance -= amount

    def get_balance(self):
        return self.__balance

acct_one = Wallet()
acct_one.deposit(3)
print(acct_one.get_balance()) # 3

acct_one.deposit(50)
print(acct_one.get_balance()) # 53

acct_one.deposit(-4)  # ValueError: Amount must be positive
acct_one.withdraw(-8) # ValueError: Amount must be positive
acct_one.withdraw(58) # ValueError: Insufficient funds
```

As you can see, the `__validate` method is private, and runs behind the scenes in the `deposit()` and `withdraw()` public methods to make sure the amount is always valid.

In a coming lesson, you will learn more about how attributes prefixed with a double underscore works.

In summary, encapsulation locks down internal data behind clear public methods. That's how you keep your classes safe from tampering and centralize validation in one place. You can update or extend your code freely, knowing that outside code only touches the interfaces you expose.

What Is Inheritance and How Does It Promote Code Reuse?

Inheritance is the next key concept of object-oriented programming (OOP) we'll cover.

Let's take a deeper look at this concept and how it lets you write reusable code.

With inheritance, a subclass (or child class) can use the attributes and methods of a base class (or parent class). This allows you to reuse code, create clear class hierarchies, and customize behavior without rewriting everything. You can customize by extending existing methods or overriding them in the child class.

Here's the basic syntax for inheritance:

```
class Parent:
    # Parent attributes and methods


class Child(Parent):
    # Child inherits, extends, and/or overrides where necessary
```

For the `Child` class to inherit from the `Parent` class, you have to pass the Parent to the Child.

This style is called single inheritance, since a child class inherits from exactly one parent class.

Here's an example:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return f'{self.name} makes a sound'

class Dog(Animal):
    bark = 'woof! woof!! woof!!!'

jack = Dog('Jack')
print(jack.sound())  # Jack makes a sound
print(jack.bark)  # woof! woof!! woof!!!
```

You can see that we're able to reuse the `self.name` attribute and the `sound()` method from the parent `Animal` class in the child `Dog` class.

Let's override the `sound()` method from the parent `Animal` class in the child `Dog` class so we can have `sound()` use the `bark` class variable:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return f'{self.name} makes a sound.'

class Dog(Animal):
```

```
    bark = 'woof! woof!! woof!!!'

    # Override sound() to use bark class variable
    def sound(self):
        return f'{self.name} barks {self.bark}'

jack = Dog('Jack')
print(jack.sound())  # Jack barks woof! woof!! woof!!!
```

If you want to keep the return value of `sound()` and add the bark class variable later, you can extend `sound()` by using the `super()` function:

```
class Animal:
    def __init__(self, name):
        self.name = name

    def sound(self):
        return f'{self.name} makes a sound'

class Dog(Animal):
    bark = 'woof! woof!! woof!!!'

    # Call Animal.sound(), then append bark
    def sound(self):
        base = super().sound()
        return f'{base}, then {self.name} barks {self.bark}'

jack = Dog('Jack')
print(jack.sound())  # Jack makes a sound, then Jack barks
woof! woof!! woof!!!
```

In this example, `base` is the result of calling the `sound()` method from the `Animal` class, and then we append the `Dog` class's specific sound to it. This way, you can extend the functionality of the parent `Animal` class while still keeping its original behavior.

There's also multiple inheritance, where a child class can inherit from more than one parent class.

Here's the basic syntax of multiple inheritance:

```
class Parent:
    # Attributes and methods for Parent

class Child:
    # Attributes and methods for Child

class GrandChild(Parent, Child):
    # GrandChild inherits from both Parent and Child
    # GrandChild can combine or override behavior from each
```

A simple way to demonstrate multiple inheritance is with a frog, which can both walk on land and swim in water:

```
class Walker:
    def walk(self):
        return 'I can walk on land'

class Swimmer:
    def swim(self):
        return 'I can swim in water'

# Amphibian inherits from both Walker and Swimmer
class Amphibian(Walker, Swimmer):
    def __init__(self, name):
```

```
        self.name = name


    def introduce(self):
        return f"I'm {self.name} the frog. {self.walk()} and
{self.swim()}."


frog = Amphibian('Freddy')
print(frog.introduce())
# Output: I'm Freddy the frog. I can walk on land and I can
swim in water.
```

What Is Polymorphism and How Does It Promote Code Reuse?

Polymorphism is the next key concept of object-oriented programming (OOP) we will talk about.

With polymorphism, you have access to an interface where you can interact with many objects of the same kind.

Let's take a deeper look at polymorphism and how it lets you reuse code.

Polymorphism allows methods in different classes to share the same name but perform different tasks. You call the same method name on different objects, and each responds in its own way.

Here's the basic example of polymorphism:

```
class A:
   def action(self): ...


class B:
   def action(self): ...
```

```
class C:
    def action(self): ...

Class().method()  # Works for A, B, or C
```

Here's an example using different animal sounds to depict polymorphism:

```
class Cat:
    def speak(self):
        return "A cat meow"

class Bird:
    def speak(self):
        return "A bird tweet"

class Monkey:
    def speak(self):
        return "A monkey ooh ooh aah aah ooh ooh aah aah"

def animal_sound(animal):
    print(animal.speak())

animal_sound(Cat())
animal_sound(Bird())
animal_sound(Monkey())
```

In this example, `animal_sound()` is a function that takes any object with a `speak()` method.

When you pass in a `Cat`, `Bird`, or `Monkey`, it calls the `speak()` method of the object and prints the result. Because each class defines `speak()` differently,

you get different outputs from the same function. That's polymorphism in action.

Here's another example, this time with instances and an attribute:

```
class Twitter:
    def __init__(self, content):
        self.content = content

    def post(self):
        return f"🐦 Tweet: '{self.content}' (280 chars max)"

class Instagram:
    def __init__(self, content):
        self.content = content

    def post(self):
        return f"📸 Instagram Post: '{self.content}' + ✨ filters"

class LinkedIn:
    def __init__(self, content):
        self.content = content

    def post(self):
        return f"💼 LinkedIn Article: '{self.content}' (Professional Mode)"

def start(social_media):
    print(social_media.post())  # Calls .post() on any object
```

```
# Instances
tweet = Twitter('Just learned Python polymorphism!')
photo = Instagram('Sunset vibes 🌅')
article = LinkedIn('Why OOP matters in 2024')


# The polymorphic calls - same function, different outputs
start(tweet) # 🐦 Tweet: 'Just learned Python polymorphism!' (280 chars max)
start(photo) # 📸 Instagram Post: 'Sunset vibes 🌅' + ✨ filters
start(article) # 💼 LinkedIn Article: 'Why OOP matters in 2024' (Professional Mode)
```

There's also a kind of polymorphism called **inheritance-based polymorphism**.

In inheritance-based polymorphism, a parent class defines a method, and multiple child classes override that method in their own way. You can then call the same method on any child object, and it behaves differently depending on which child class it is.

Here's an example:

```
class Animal:
    def speak(self):
        return 'Some generic sound'


class Cat(Animal):
    def speak(self):
        return 'A cat meow'


class Dog(Animal):
    def speak(self):
```

```
        return 'A dog barks woof woof'

class Monkey(Animal):
    def speak(self):
        return 'A monkey ooh ooh aah aah ooh ooh aah aah'

print(Cat().speak()) # A cat meow
print(Dog().speak()) # A dog barks woof woof
print(Monkey().speak()) # A monkey ooh ooh aah aah ooh ooh aah aah
print(Animal().speak()) # Some generic sound
```

You can see that each child class of the parent `Animal` class overrides the `speak()` method to provide its own implementation. So when you call the `speak()` method on an instance of each subclass, it returns the specific sound associated with that animal.

You can also take things further and do the calling in a list, then loop through the list to display what the `speak()` method returns for each:

```
animals = [Cat(), Dog(), Monkey()]

for animal in animals:
    print(animal.speak())

# Output:
# A cat meow
# A dog barks woof woof
# A monkey ooh ooh aah aah ooh ooh aah aah
```

What is Name Mangling and How Does it Work?

In a previous lesson, you learned about prefixing attributes with a single underscore and a double underscore.

To remind you of the difference between them, a single underscore is a convention that means the attribute is meant for internal use in the class and should not be directly accessed from outside the class. Double underscore, on the other hand, prevents that attribute from being accessed directly from outside the class.

Here's an example that demonstrates how the two work:

```
class Example:
    def __init__(self):
        self._internal = 'I can be accessed from outside the class, but should not'
        self.__private = 'You cannot access me directly from outside the class'


obj = Example()


print(obj._internal) # I can be accessed from outside the class, but should not
print(obj.__private)  # AttributeError: 'Example' object has no attribute '__private'
```

Prefixing an attribute with a double underscore triggers Python's name mangling process, in which Python internally renames the attribute by adding an underscore and the class name as a prefix, turning `__attribute` into `_ClassName__attribute`.

To see this in action, you create an instance of the class and use the `__dict__` special attribute of that instance, which is a dictionary containing the object's attributes:

```
class Example:
    def __init__(self, internal, private):
        self._internal = internal
```

```
        self.__private = private


example1 = Example(
    'I can be accessed from outside the class, but should not',
    'I cannot be accessed directly from outside the class'
)


print(example1.__dict__)
```

The result would be:

```
{
  '_internal': 'I can be accessed from outside the class, but should not',
  '_Example__private': 'I cannot be accessed directly from outside the class'
}
```

As you can see, the `__private` attribute is stored as `_Example__private`. This means you can still access that attribute outside the class this way:

```
class Example:
    def __init__(self, internal, private):
        self._internal = internal
        self.__private = private


example1 = Example(
    'I can be accessed from outside the class, but should not',
    'I cannot be accessed directly from outside the class'
)
example2 = Example(
```

```
    'I should not be accessed from outside the class',
    'But I can be accessed from outside the class with name mangling'
)


print(example1._Example__private) # I cannot be accessed directly from outside the class
print(example2._Example__private) # But I can be accessed from outside the class with name mangling
```

So, why does Python do name mangling?

The main purpose of name mangling is to prevent accidental attribute and method overriding when you use inheritance. Here's an example that makes that clear:

```
class Parent:
    def __init__(self):
        self.__data = 'Parent data'


class Child(Parent):
    def __init__(self):
        super().__init__()
        self.__data = 'Child data'


c = Child()
print(c.__dict__) # {'_Parent__data': 'Parent data', '_Child__data': 'Child data'}
```

You can see that both the `Parent` class and the `Child` that inherits from it have their separate `_class__data` attributes. This is made possible with name mangling. Otherwise, the `Child` would have overwritten the Parent data by accident.

Here's what would have happened without allowing Python to do the name mangling, that is if you don't prefix the attributes in both classes with double underscore:

```
class Parent:
    def __init__(self):
        self.data = 'Parent data'


class Child(Parent):
    def __init__(self):
        super().__init__()
        self.data = 'Child data'


c = Child()
print(c.__dict__)  # {'data': 'Child data'}
```

So, which should you use to prefix attributes between single underscore (_) and double underscore (__)? It depends. If an attribute is only meant for internal use within the class, stick with a single underscore.

But if you're working with a class that will be inherited, you should use a double underscore so the attribute from the parent doesn't get overridden.